CYBERTRACE-AI: AI-POWERED DIGITAL EVIDENCE RECONSTRUCTION

A PROJECT REPORT

Submitted by

ARILLI JAGADEESH A (241901502)

SACHIN KRISHNA B (241901507)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institute

CHENNAI

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**CYBERTRACE AI**" is the bonafide work of **" Arilli Jagadeesh A (2116241901502) and Sachin Krishna B (211624191507)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Bhuvana S, Ph.D.,

ASSISTANT PROFESSOR(SG)

Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

CyberTrace AI is an intelligent digital forensic platform designed to automate the reconstruction and analysis of complex cyberattacks by correlating digital evidence from multiple sources, including Windows Event Logs (EVTX), Linux authentication logs, CSV files, and JSON audit records. The system uses SHA-256 hashing to maintain evidence integrity and support chain-of-custody verification. It normalizes collected events into a unified format and represents relationships between users, hosts, IP addresses, processes, and system artifacts through a directed, weighted Knowledge Graph. An A* search-based pathfinding engine with a privilege-gap heuristic identifies the most probable attack paths while reducing unnecessary graph traversal and investigation time. A Constraint Satisfaction Problem (CSP) solver analyzes chronological event sequences, detects timestamp anomalies, identifies broken execution chains, and reconstructs missing process relationships through virtual state restoration. The platform includes a responsive web interface developed using React 19, TypeScript, Vite, and Tailwind CSS, with a FastAPI backend and PostgreSQL database. It also integrates a Retrieval-Augmented Generation (RAG) conversational assistant to help investigators explore evidence and understand attack sequences, along with an automated reporting engine that generates detailed forensic reports in PDF and DOCX formats. By combining evidence integrity verification, graph-based reasoning, timeline validation, and AI-assisted investigation, CyberTrace AI aims to reduce manual effort, minimize false positives, accelerate incident response, and provide explainable, verifiable findings for cybersecurity operations and digital forensic investigations.

Keywords: Digital Forensics, Cyber Incident Response, Knowledge Graph, A,A* Pathfinding, Constraint Satisfaction Problems, Forward Chaining Inference, Timeline Reconstruction, Chain of Custody.

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor(SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

1. ARILLI JAGADEESH A (241901502)

2. SACHIN KRISHNA B (241901507)

INDEX

S.NO	TABLE OF CONTENT	PAGE NUMBER
	ABSTRACT	III
1	**INTRODUCTION**	1
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	4
	1.6 SDG ALIGNMENT	5
2	**LITERATURE REVIEW**	6
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	6
	2.2 RESEARCH GAPS &VULNERABILITIES	6
	2.3 PROPOSED SYSTEM	8
3	**SYSTEN DESIGN**	9
	3.1 SYSTEM ARCHIETURE	9
	3.2 MODULE DESIGN	11
	3.3 DATA FLOW/UML DIAGRAMS	11
	3.4 ENTITY-RELATIONSHIP DIAGRAM	13
	3.5 USE-CASE DIAGRAM	14
	3.6 MODULE DECOMPOSITION	15

4	**SYSTEM DESCRIPTION**	16
	4.1 DATA COLLECTION	16
	4.2 DATA PREPROCESSING	17
	4.3 DOCUMENT PROCESSING AND OCR	18
	4.4 AI-BASEDDOCUMENTVERIFICATION	20
	4.5 SCAM DETECTION AND AUTHENTICITY SCORING	21
	4.6 DATABASE AND CASE MANAGEMENT	22
	4.7 POLICE DASHBOARD AND RBAC	23
	4.8 CITIZEN PORTAL AND INTEGRATION LAYER	24
5	**CONCLUSION & FUTURE WORK**	25
	5.1 CONCLUSION	25
	5.2 LEARNINGOUTCOMES	25
	5.3 CHALLENGESFACED	26
	5.4 FUTUREENHANCEMENTS	26
	REFERENCES	27
	APPENDIX	29

LIST OF TABLES

TABLE NO	TITLE	PAGE NUMBER
1.1	PROJECTDEVELOPMENTPHASES	4
2.1	COMPARTIVE ANALYSIS OF REQUIREMENTS & ALGORITHMIC MODE	7
3.1	HARDWARE AND INFRASTRUCTURE SPECIFICATIONS	10
3.2	SOFTWARE ENVIROMENTS AND TECHNOLOGY STACK	11
4.1	DOCUMENTVALIDATIONRULES	18
4.2	OCRPROCESSINGSTAGES	20

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.1	SYSTEM ARCHITECTURE	9
3.2	MODULE DESIGN	11
3.3	CONTEXT LEVEL	12
3.4	DATA-FLOW DIAGRAM	13
3.5	ENTITY-RELATIONSHIPDIAGRAM	14
3.6	USE-CASEDIAGRAM	14
4.1	DOCUMENT PROCESSING AND OCR WORKFLOW.	19
4.2	AI-BASED VERIFICATION PIPELINE.	21
4.3	AUTHENTICITY SCORING WORKFLOW.	22
4.4	POLICE DASHBOARD WORKFLOW	23

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

The rapid growth of digital infrastructure, cloud environments, and interconnected enterprise networks has significantly increased the complexity of modern cyberattacks. Cybercriminals and advanced threat groups conduct multi-stage attacks involving credential theft, privilege escalation, lateral movement, and unauthorized access to critical systems. These activities generate enormous volumes of digital evidence across different operating systems, security tools, and network devices.

Digital Forensics and Incident Response (DFIR) plays a vital role in identifying the root cause, scope, and sequence of cyber incidents. Investigators must analyze evidence from multiple sources, including Windows Event Logs (EVTX), Linux system and authentication logs, Endpoint Detection and Response (EDR) telemetry, firewall records, and structured CSV, JSON, and XML files. Correlating these heterogeneous data sources manually is time-consuming and makes it difficult to identify relationships between events, detect manipulated timestamps, and reconstruct complete attack sequences.

Accurate digital evidence reconstruction requires maintaining evidence integrity, establishing reliable event timelines, and verifying relationships between system activities. Cryptographic hashing, chronological analysis, and chain-of-custody verification are essential to ensure that collected evidence remains trustworthy and suitable for forensic investigation. Therefore, an automated and intelligent platform is needed to correlate multi-source evidence, identify suspicious activities, reconstruct attack paths, and generate explainable forensic findings. CyberTrace AI addresses these challenges by combining graph-based analysis, intelligent pathfinding, timeline validation, and AI-assisted investigation to improve the speed, accuracy, and reliability of digital forensic investigations.

1.2 MOTIVATION

Despite significant investments in cybersecurity defenses and Security Information and Event Management (SIEM) systems, digital forensic investigations remain largely manual, time-consuming, and prone to errors. Investigators face challenges in analyzing logs from different operating systems and security tools due to variations in file formats, timestamp conventions, and entity naming standards. The enormous volume of security alerts makes it difficult to

correlate events across multiple systems and reconstruct complete attack sequences efficiently, often extending investigation times and increasing alert fatigue. Existing machine learning and large language model approaches may also produce unreliable results and lack clear, verifiable explanations of their conclusions. These challenges highlight the need for CyberTrace AI, an automated and explainable digital forensic platform that combines A* graph search, Constraint Satisfaction Problem (CSP) solving, and automated forensic reporting to reconstruct attack paths, validate event timelines, identify anomalies, and support faster, more reliable investigations.

1.3 PROBLEM STATEMENT

During security breaches, digital forensic investigators must identify the initial point of compromise, determine which user accounts and systems were affected, trace lateral movement, identify attacker tactics and techniques, and establish a reliable timeline of events. However, existing security monitoring and forensic tools often struggle to reconstruct complete attack sequences because of missing parent-process relationships, manipulated logs, inconsistent timestamps, and unsynchronized system clocks. These issues can create broken execution chains and chronological contradictions, making it difficult to establish the actual sequence of attacker activities. Conventional graph search methods may also explore numerous irrelevant nodes, increasing investigation time and computational overhead. CyberTrace AI addresses these challenges by integrating heterogeneous log ingestion, Knowledge Graph construction, A* search-based attack path reconstruction, and Constraint Satisfaction Problem (CSP) solving. The platform identifies suspicious relationships, detects timeline inconsistencies, restores missing process links through virtual state reconstruction, and generates explainable forensic findings to support accurate, efficient, and verifiable cyber incident investigations.

1.4 OBJECTIVES OF THE PROJECT

The primary objective of CyberTrace AI is to develop and validate an intelligent digital forensic platform that automates evidence collection, analysis, and cyberattack reconstruction while maintaining evidence integrity and reliability. The system uses SHA-256 hashing and metadata tracking to preserve evidence authenticity and chain of custody. It supports multiple log formats, including Windows EVTX, Linux Syslog, CSV, JSON, and XML, and converts them into a unified event format for consistent analysis. The platform constructs a directed, weighted Knowledge Graph to represent relationships between users, devices, IP addresses, processes, and system activities. An A* search engine with a privilege-gap heuristic identifies probable attack paths while reducing unnecessary graph exploration compared with Uniform

Cost Search. A Constraint Satisfaction Problem (CSP) solver validates event sequences, detects timestamp inconsistencies, and identifies broken process relationships. Additionally, a forward-chaining inference engine applies cybersecurity rules to identify suspicious activities, including brute-force attacks, remote execution, and ransomware-related backup tampering. These capabilities aim to improve investigation speed, accuracy, transparency, and the reliability of digital forensic findings.

1.2 SCOPE AND BOUNDARIES

CyberTrace AI supports the ingestion and normalization of Windows EVTX logs, Linux Syslog and authentication logs, CSV, JSON, JSONL, and XML files. The platform includes Knowledge Graph construction, A* and Uniform Cost Search algorithms, CSP-based timeline validation, and forward-chaining security rule evaluation to reconstruct attack paths and identify suspicious activities. Security features include role-based access control for administrators, investigators, analysts, and viewers, bcrypt password hashing, and JWT-based authentication. Investigation tools include an interactive graph visualizer, chronological event timeline, CSP validation reports, an AI-powered assistant, and exportable forensic reports. The system is designed for containerized deployment using Docker, Docker Compose, PostgreSQL 15, and Nginx.

Direct acquisition and analysis of volatile memory images, kernel-level rootkit detection, deep packet inspection of encrypted network traffic, and large-scale distributed log processing using technologies such as Apache Kafka and Apache Flink are outside the current project scope and may be considered for future enhancements.

TABLE 1.1. PROJECT DEVELOPMENT PHASES

Phase	Phase Name	Core Activities & Objectives	Key Deliverables
Phase I	**Requirements Analysis & Feasibility Study**	Literature review on DFIR standards (NIST SP 800-86); identification of limitations in existing SIEMs; formulation of system requirements and mathematical problem statements.	Project Charter, PRD, TRD, System SRS documentation.
Phase II	**Architectural & Database Design**	Definition of micro-modular system architecture; schema design for PostgreSQL (Postgres 15 DDL, GIN indexing); API endpoint specification; RBAC model definition.	High-Level Design (HLD), ER Diagrams, schema.sql, and seed.sql.
Phase III	**Evidence Ingestion & Normalizer Pipeline**	Implementation of modular parsers (EVTX, Syslog, CSV, JSON, XML); development of UTC timestamp converters; SHA-256 integrity verification modules.	backend/app/parsers/, backend/app/normalization/, and parser test suites.

Phase IV	**Knowledge Graph & Symbolic AI Engineering**	Implementation of entity/relationship extraction algorithms; mathematical modeling of A∗A *A∗ and UCS search engines; implementation of CSP backtracking engine and Forward Chaining expert rules.	astar_search.py, uniform_cost_search.py, backtracking_engine.py, forward_chaining.py.
Phase V	**Frontend UI & Full-Stack Integration**	Development of React 19 single-page dashboard (Vite, TypeScript, Tailwind CSS); Zustand state store; integration with FastAPI REST endpoints; PDF/DOCX report generation.	Complete frontend/src/ pages, ReportLab PDF generator, and python-docx engine.

1.3 SECURITY REQUIREMENTS & CONSIDERATIONS

CyberTrace AI ensures data protection and privacy by encrypting data during transmission and storage, minimizing the collection of personally identifiable information, and validating inputs to prevent data poisoning. The platform incorporates security measures to protect AI components against adversarial manipulation, evasion, and prompt-injection attacks, supported by regular security testing. Role-based access control and least-privilege policies restrict access to sensitive logs, model configurations, and API credentials, while secure inter-service communication, token rotation, and rate limiting strengthen API security. Continuous monitoring and audit logging track administrative actions, model changes, and unusual system activities. The platform also emphasizes compliance, governance, and explainability through audit trails, data-flow documentation, model documentation, and alignment with relevant security frameworks. Explainable AI techniques and human oversight help investigators understand flagged events and validate critical decisions before taking remediation actions.

1.4 SDG Alignment of CyberTrace-AI

CyberTrace AI contributes to Sustainable Development Goal 9 by introducing an AI-driven digital forensic platform that strengthens digital infrastructure, improves cybersecurity resilience, and encourages the adoption of innovative technologies such as machine learning, vector search, and automated reporting. It supports Sustainable Development Goal 16 by enabling faster, evidence-based cybercrime investigations, promoting accountability, supporting justice, and helping protect critical information systems from malicious activities. The project also aligns with Sustainable Development Goal 17 by integrating open-source technologies such as FastAPI, React, and PostgreSQL, encouraging collaboration among cybersecurity professionals, academic institutions, and industry organizations to exchange threat intelligence, share knowledge, and improve cybersecurity practices.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEMS WITH SECURITY APPROACHES

Existing cybersecurity platforms provide log collection, event correlation, anomaly detection, and intrusion prevention through different approaches. Splunk Enterprise Security centralizes and normalizes security logs for rule-based correlation, while Darktrace uses AI to identify deviations from normal network behaviour. Microsoft Sentinel provides cloud-based security monitoring and automated response, and IBM QRadar correlates log and network-flow data to detect potential threats. Snort and Suricata identify suspicious network traffic using signature-based intrusion detection. These platforms incorporate security controls such as encrypted communication, role-based access control, audit logging, secure data collection, and automated threat response. However, reconstructing complete attack paths across heterogeneous evidence sources, validating chronological consistency, and resolving broken process relationships remain important challenges. CyberTrace AI addresses these gaps by combining multi-format log normalization, Knowledge Graph-based analysis, A* pathfinding, CSP-based timeline validation, and explainable threat reasoning within a unified digital forensic investigation platform.

2.2 RESEARCH GAPS & VULNERABILITIES

Cybersecurity and AI-driven forensic systems face several challenges, including adversarial data poisoning, limited explainability, insecure integration of external large language models, insufficient access control for sensitive evidence, and model drift. Attackers may manipulate log entries or threat intelligence data to influence analysis results, causing missed detections or false positives. Black-box AI decisions can make it difficult for investigators to verify findings and justify conclusions during audits or legal proceedings. External LLM services may introduce risks such as credential exposure, prompt injection, service outages, and third-party dependency vulnerabilities. Inadequate fine-grained access controls can also compromise evidence provenance and audit trails, while model drift may reduce detection accuracy as new data patterns emerge. CyberTrace AI aims to address these challenges through evidence integrity verification, explainable graph-based reasoning, secure access control, detailed audit logging, input validation, and continuous monitoring to support reliable and trustworthy forensic investigations.

Table 2.1 Comparative Analysis of Recruitment Systems & Algorithmic Models

System / Model	Core Algorithmic Approach	Key Strengths	Main Weaknesses / Risks	Typical Use-Case
ATS (Applicant Tracking System) – rule-based filtering	Keyword / Boolean matching, rule-based scoring	Simple to implement; transparent logic; easy to audit	Ignores context, prone to "keyword stuffing"; high false-negative rate for non-standard CVs	Small-to-mid-size firms that need basic resume triage
Classic Machine-Learning Resume Screener (e.g., logistic regression, SVM)	Supervised model trained on historical hire data (features: experience, education, skill tags)	Learns from past hiring patterns; can improve over time; quantitative ranking	Risk of perpetuating historic bias; requires large, clean training set; limited explainability	Companies with sizable historic hiring data looking to automate initial shortlist
Deep-Learning Text Embedding +Similarity Matching (e.g., BERT-based semantic similarity)	Contextual language models generate embeddings; similarity scores match job description ↔ CV	Captures semantic meaning, reduces keyword-gaming; works across languages	Compute-heavy; harder to interpret decisions; may still surface hidden biases in training corpora	Large enterprises or platforms (e.g., LinkedIn) that process millions of profiles
AI-Driven Predictive Hiring Platform (e.g., TalentScore, HireVue)	Ensemble of ML + psychometric / video-analysis models (speech, facial cues)	Predicts performance / cultural fit; combines multiple data sources	Ethical & privacy concerns; high regulatory scrutiny; potential for discrimination (gender, age)	High-growth tech firms using data-driven talent acquisition pipelines

2.3 PROPOSED HIREAI SYSTEM

2.3.1 Overview of the Proposed System

CyberTrace AI is designed as an end-to-end digital forensic investigation platform that securely ingests evidence from multiple log sources and formats. Its modular architecture includes a data ingestion layer for collecting Windows EVTX, Linux Syslog, CSV, JSON, JSONL, and XML files; a preprocessing layer for parsing, validating, and normalizing events; and a core reasoning engine that combines Knowledge Graph construction, A* search, CSP-based timeline validation, and forward-chaining security rules. The decision and explainability layer provides interactive graph visualizations, chronological event timelines, investigation dashboards, an AI-powered assistant, and detailed forensic reports. The platform can be deployed on-premises using Docker Compose and Nginx, with PostgreSQL as the database, while optional integration with external or locally hosted large language models supports evidence exploration and investigation assistance.

2.3.2 Objectives of the Proposed System

CyberTrace AI aims to reduce digital forensic investigation time through automated evidence analysis and attack-path reconstruction. It improves investigation accuracy by correlating heterogeneous logs, identifying suspicious activities, and validating chronological event sequences. The platform promotes transparency and accountability by providing explainable threat analysis, evidence provenance tracking, and audit-ready forensic reports. It protects sensitive information through encryption, secure authentication, role-based access control, and support for on-premises deployment. Its modular architecture also enables future integration of additional log parsers, security rules, graph-analysis algorithms, and AI models without requiring major changes to the existing system.

Key Advantages of Proposed Architecture

CyberTrace AI uses a modular, scalable architecture in which individual processing components can be scaled independently to handle increasing volumes of digital forensic evidence. Its hybrid AI and human-in-the-loop approach provides investigators with explainable attack-path analysis and allows them to review, validate, and override automated findings when necessary. The platform incorporates security validation and rule-based reasoning to identify suspicious activities, reduce false positives, and improve investigation reliability while keeping critical forensic decisions under human supervision.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

CyberTrace AI is engineered as a multi-tier, micro-modular enterprise forensic analysis system. The architecture cleanly separates user presentation, API orchestration, computational reasoning engines, and data persistence to ensure high throughput, horizontal scalability, and strict evidential integrity.

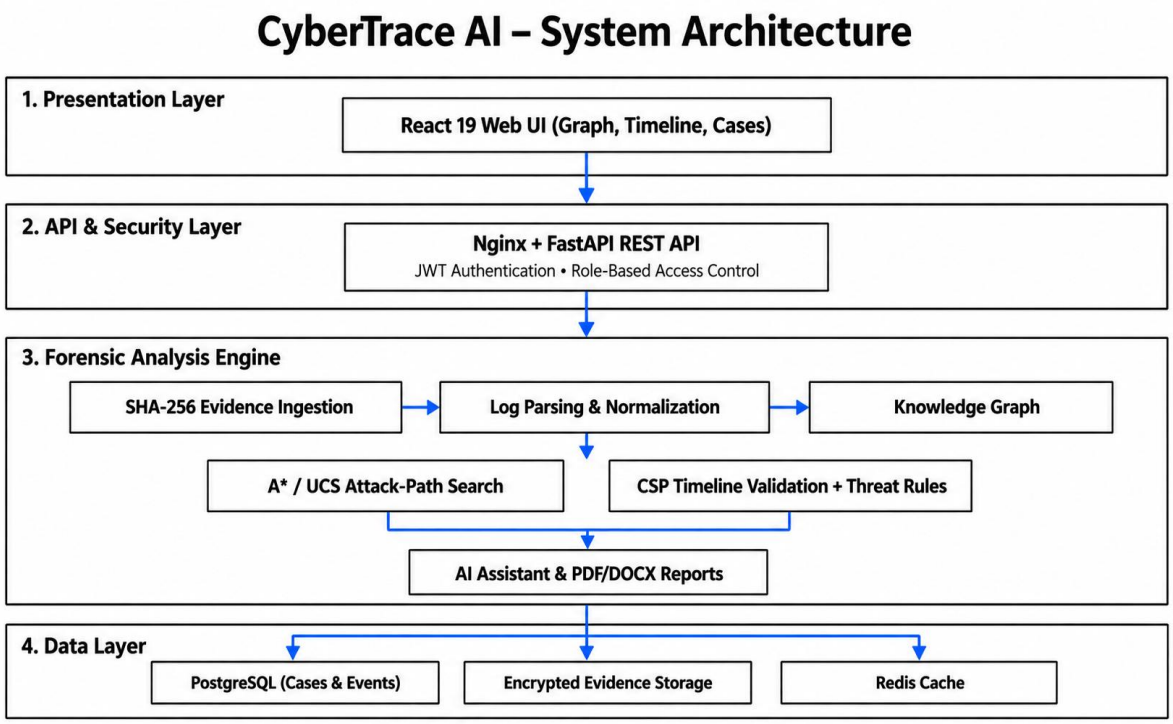


FIGURE NO. 3.1 SYSTEM ARCHITECTURE

1. Presentation Tier: Developed using React 19, Vite, TypeScript, Zustand, TanStack Query, and Tailwind CSS to provide an interactive interface for forensic investigations, authentication, session management, and data visualization.
2. API Gateway and Orchestration Tier: Uses FastAPI with Uvicorn to handle API requests, evidence uploads, input validation through Pydantic v2, CORS policies, JWT authentication, and Role-Based Access Control (RBAC).
3. Core Forensic and AI Engine Tier: Performs SHA-256 evidence hashing, parses Windows EVTX, Linux Syslog, CSV, JSON, and XML files, normalizes timestamps to UTC, constructs Knowledge Graphs, and uses CSP-based backtracking to validate timelines and reconstruct missing process relationships.
4. Data Persistence Tier: Uses PostgreSQL 15 to store cases, events, and forensic metadata

with JSONB storage, GIN indexes, fuzzy text search, and relational constraints. Redis provides in-memory caching, session management, and API rate limiting.

3.1.1 Hardware and Software Specifications

Table 3.1 Hardware and Infrastructure Specifications

Component	Specification	Rationale
Compute (CPU)	2 × Intel Xeon E5-2690 v4 (12 cores each) or AMD EPYC 7352 (24 cores)	Sufficient for parallel log parsing, vector-store indexing, and API serving.
Compute (GPU)	1 × NVIDIA A100 40 GB (or 2 × RTX 4090 24 GB for on-prem)	Accelerates LLM inference, FAISS-GPU indexing, and video-analysis models.
Memory (RAM)	128 GB DDR4 ECC	Handles large in-memory data frames (hundreds of K log events) and model weights.
Security	TPM 2.0, hardware-based full-disk encryption (AES-256), firewall rules limiting inbound traffic to 443/22	Meets data-integrity and compliance requirements.

Table 3.2 Software Environment and Technology Stack

Layer	Technology / Tool
Operating System	Ubuntu 24.04 LTS (or Windows Server 2022 for on-prem)
Container Runtime	Docker 26.0 + containerd 1.7
Orchestration	Kubernetes 1.30 (EKS/GKE/AKS)
Frontend	React 19, Vite 5, TypeScript 5.4, Tailwind CSS 3, shadcn/ui 0.9
Programming Language	Python 3.12.4
Database (SQL)	PostgreSQL 15.5 + PostGIS 3.4
Cache / Message Queue	Redis 7.2 (with Redis-JSON)
Vector Store	FAISS-GPU 1.8.0 (or Milvus 2.4)

3.2 MODULE DESIGN

The system follows a layered microservices architecture consisting of data ingestion, preprocessing and feature extraction, embedding and video analysis, a core scoring engine, fairness and explainability modules, and a dashboard layer. The backend is developed using FastAPI, while React 19 provides the frontend interface. FAISS-GPU enables efficient vector similarity search, and Kubernetes manages container orchestration and horizontal scaling to support increasing workloads.

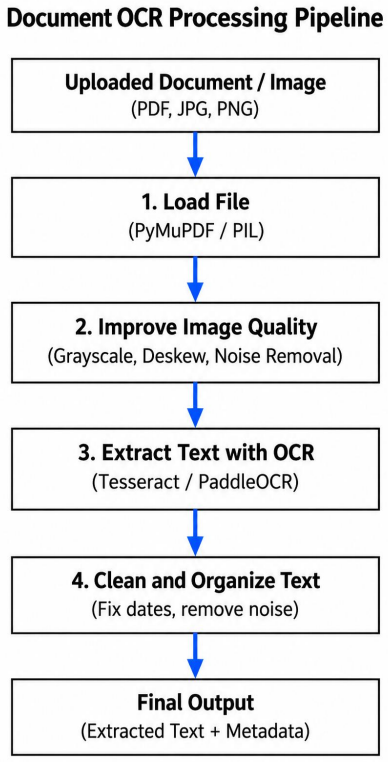


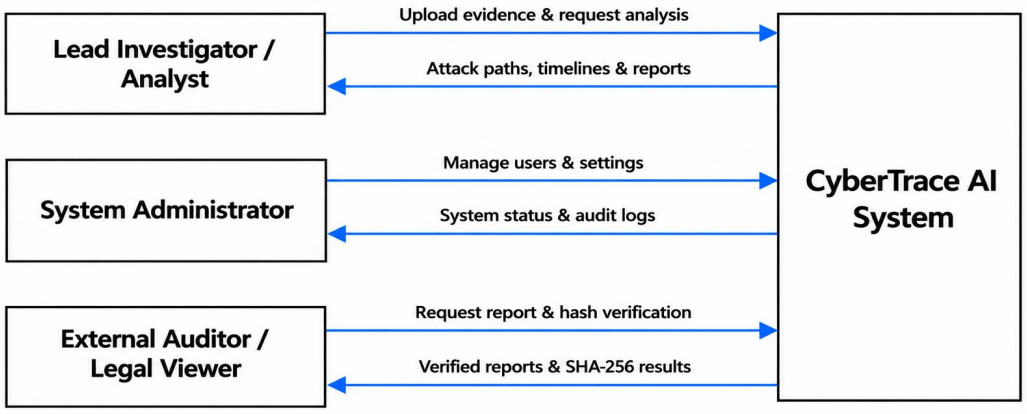
FIGURE NO.3.2 MODULEDESIGN

3.3 Data-Flow Diagrams

Data-Flow Diagrams (DFDs) illustrate how digital evidence, normalized security logs, Knowledge Graphs, and forensic reports move through CyberTrace AI. They describe the interaction between external users, system processes, and data storage components, providing a clear overview of how information is collected, processed, analyzed, and presented.

The Level-0 Data-Flow Diagram represents CyberTrace AI as a single central system and defines its interaction with external entities, including the Lead Investigator, Forensic Analyst, System Administrator, and External Auditor. The Lead Investigator initiates investigations and reviews attack reconstruction results, while the Forensic Analyst uploads evidence, examines event timelines, and analyzes suspicious activities. The System Administrator

manages user accounts, access permissions, and system configurations. The External Auditor reviews forensic reports, evidence integrity records, and audit trails to verify the reliability and traceability of investigative findings. CyberTrace AI processes the incoming requests and evidence, performs forensic analysis, and returns investigation results, validated timelines, security alerts, and reports to the appropriate users.



Level-1 Data-Flow Diagram

The Level-1 DFD decomposes Process 0 into its seven primary sub-processes, mapping incoming log streams to intermediate data stores and final report outputs.

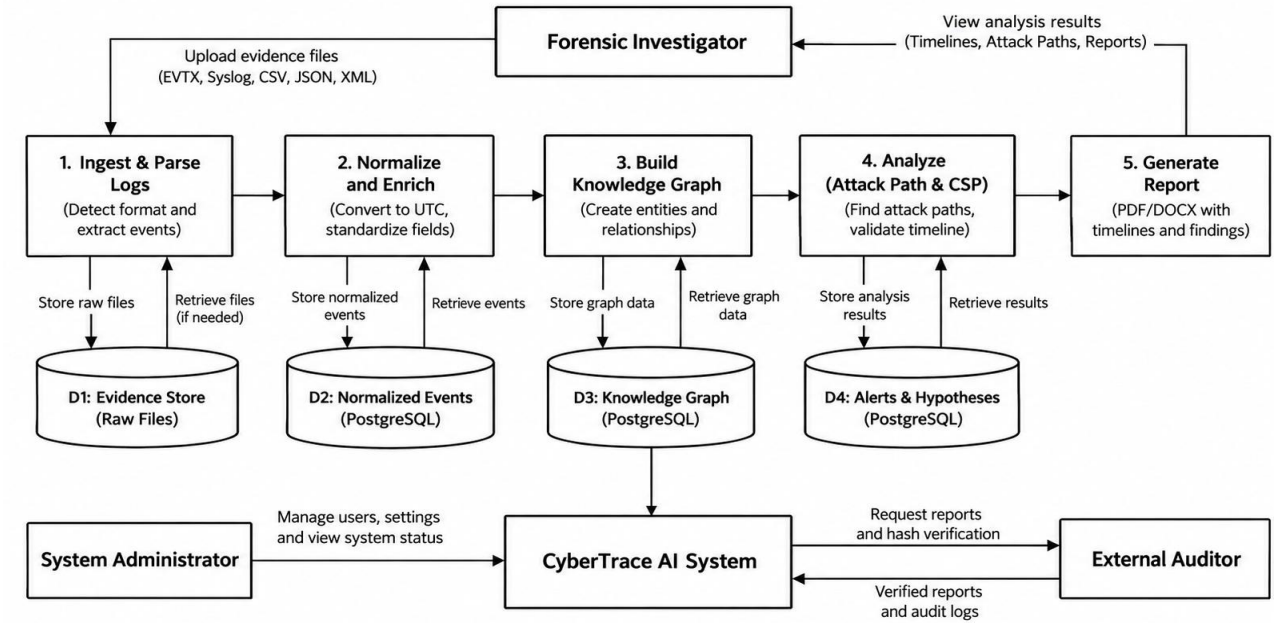


FIGURE NO.3.4 Data-Flow Diagram

3.4 Entity-Relationship Diagram

CyberTrace AI uses PostgreSQL 15 as its relational database to maintain data integrity,

prevent duplicate evidence records, and optimize forensic queries through indexing.

The users table stores investigator accounts, Bcrypt password hashes, and authorization roles such as admin, investigator, analyst, and viewer. The cases table maintains incident titles, descriptions, priorities, and links to the responsible investigator. The evidence table records uploaded files, original and sanitized filenames, file sizes, storage paths, parsing status, and SHA-256 checksums to verify evidence integrity. The events table stores normalized forensic events, including timestamps, user identities, hostnames, IP addresses, processes, severity levels, and raw JSONB data, supported by GIN indexing for efficient searches. The alerts table records security rule triggers and MITRE ATT&CK detections, including rule names, confidence scores, and acknowledgement status. The ai_reports table stores case-specific forensic findings, executive summaries, root cause analysis, risk levels, structured recommendations, and PDF/DOCX report paths.

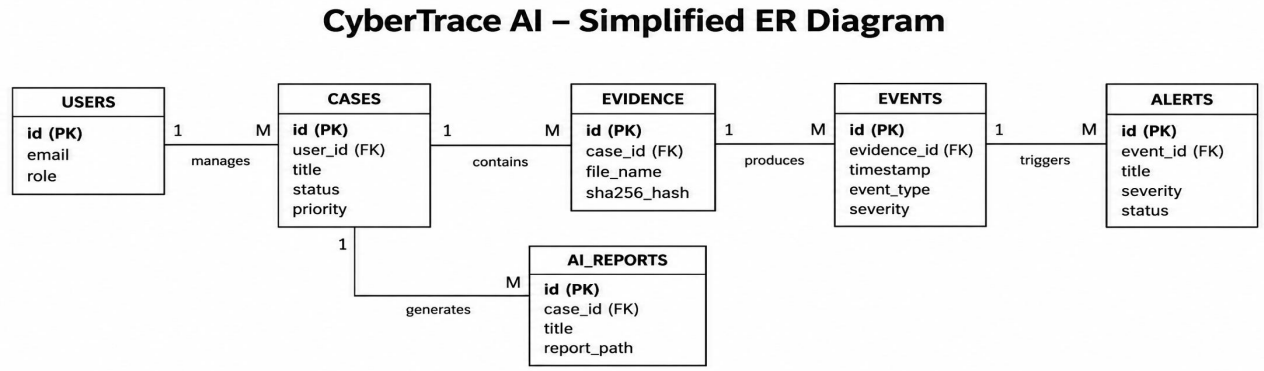


FIGURE NO. 3.5 Entity-Relationship Diagram

3.5 Use-Case Diagram

The use-case diagram illustrates the interactions between different users and the CyberTrace AI platform. It identifies four primary actors: Administrator, Forensic Analyst, Auditor / Legal Viewer, and Lead Investigator. The Administrator manages user accounts, roles, authentication, and system audit logs. The Forensic Analyst creates and manages forensic cases, uploads digital evidence, verifies SHA-256 hashes, and uses the AI assistant to analyze suspicious activities. The Lead Investigator reconstructs attack paths using A* and Uniform Cost Search (UCS), explores the Knowledge Graph, and validates event timelines using Constraint Satisfaction Problem (CSP) techniques. The Auditor / Legal Viewer accesses forensic reports and verifies evidence integrity for auditing and legal purposes.

The diagram also represents relationships between system functions. Evidence uploading includes log parsing and normalization to prepare data for forensic analysis. Attack-path reconstruction can be extended through Knowledge Graph exploration, while timeline

validation supports the generation and export of forensic reports in PDF and DOCX formats. Overall, the use-case diagram provides a clear overview of the platform's functional requirements, user responsibilities, and interactions, demonstrating how CyberTrace AI supports secure evidence handling, automated investigation, and reliable forensic reporting.

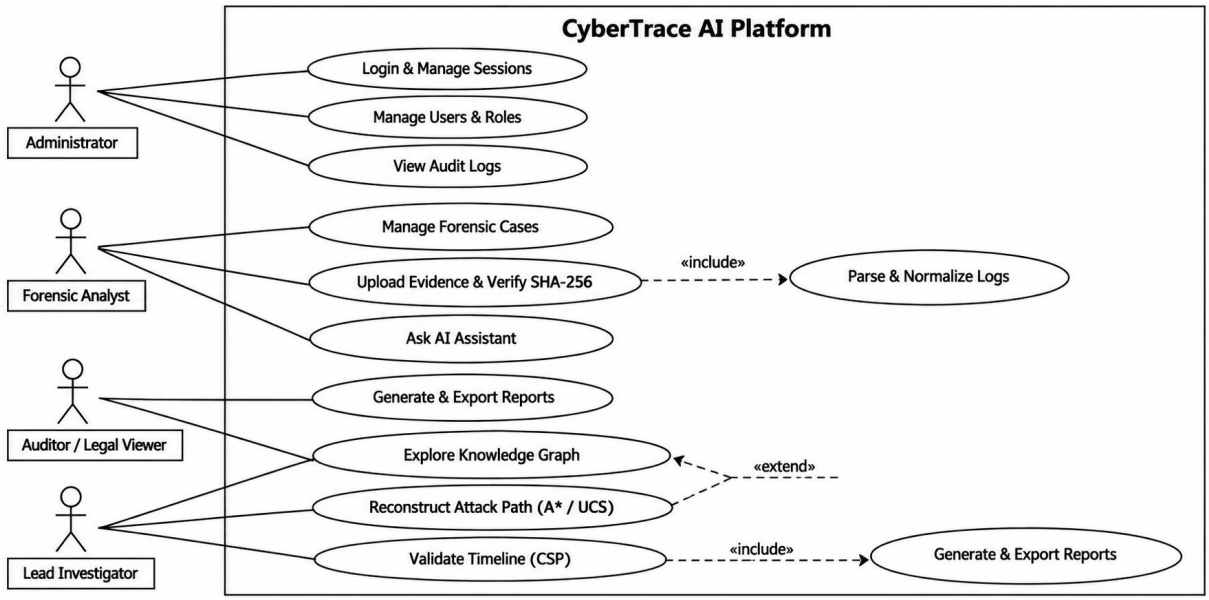


FIGURE NO.3.6 Use-Case Diagram

1. UC-4 (Ingest Evidence and Verify SHA-256) allows investigators to upload digital forensic files in supported formats. The system calculates a SHA-256 checksum for each uploaded file, checks for duplicate evidence, records relevant metadata, and initiates the log parsing and normalization process (UC-5). This helps maintain evidence integrity and supports chain-of-custody verification.
2. UC-7 (Reconstruct Attack Path) enables investigators to identify the initial point of compromise and the target asset within the Knowledge Graph. The system uses A* search or Uniform Cost Search (UCS) to identify the most suitable path between the selected entities and displays the reconstructed attack sequence for further investigation.
3. UC-8 (Run CSP Validation and Backtracking) validates the chronological order and structural relationships of forensic events. The system identifies timestamp inconsistencies, detects broken process relationships, and attempts to resolve orphan process lineages through constraint-based backtracking. It produces validation results that help investigators assess timeline consistency.
4. UC-10 (Generate and Export Reports) combines investigation findings, reconstructed attack paths, timeline validation results, evidence integrity details, and mitigation

recommendations into comprehensive forensic reports. The reports can be exported in PDF and DOCX formats to support documentation, auditing, and legal review.

3.6 Module Decomposition

1. CyberTrace AI follows a modular software architecture in which each subsystem performs a specific task and communicates through well-defined interfaces. This design simplifies testing, maintenance, debugging, and future enhancements.
2. Module 1.0 – Evidence Ingestion and Parsing: Handles forensic file uploads, storage, and SHA-256 checksum verification. It uses a parser factory to extract events from Windows EVTX, Linux Syslog, CSV, JSON, and XML files.
3. Module 2.0 – Semantic Normalization: Converts different timestamp formats into a standardized UTC format and maps inconsistent log fields into a common structure for accurate event correlation.
4. Module 3.0 – Knowledge Graph and Topology: Identifies users, hosts, IP addresses, and processes from normalized events. It creates relationships between these entities, removes duplicate nodes, and assigns edge weights based on event severity and anomaly scores.
5. Module 4.0 – Symbolic AI and Reasoning: Implements A* search and Uniform Cost Search (UCS) for attack-path reconstruction, CSP backtracking for timeline validation, and forward-chaining rules for identifying suspicious activities.
6. Module 5.0 – Reporting and AI Assistant: Combines verified forensic findings into PDF and DOCX reports and prepares relevant evidence context for the Retrieval-Augmented Generation (RAG) assistant to answer investigation-related queries.
7. Module 6.0 – Presentation and Frontend: Provides a responsive React 19 interface for case management, evidence uploads, Knowledge Graph visualization, timeline analysis, validation results, and forensic report access.

CHAPTER 4

IMPLEMENTATION

4.1 User Authentication and Access Control

User authentication and authorization in CyberTrace AI are implemented using an asynchronous, token-based security architecture designed to prevent unauthorized access to sensitive forensic evidence and case materials.

4.1.1 Cryptographic Password Hashing

All user credentials stored in the PostgreSQL database are protected using the salted Bcrypt algorithm via passlib[bcrypt]. Bcrypt uses an adaptive work factor (cost parameter set to 12), ensuring that password hashes resist brute-force dictionary attacks, rainbow table matching, and hardware-accelerated cracking. Passwords are never logged or stored in plaintext.

python

```
# Implementation excerpt from backend/app/core/security.py
from passlib.context import CryptContext
from datetime import datetime, timedelta
from jose import jwt
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)
```

4.1.2 OAuth2 JSON Web Token (JWT) Lifecycle

CyberTrace AI uses JWT-based authentication to secure access to its APIs. The client submits the user's email and password to the /api/v1/auth/login endpoint. The server verifies the email and compares the submitted password against the stored Bcrypt password hash. If the credentials are valid, the server generates a JSON Web Token (JWT) signed using HMAC-SHA256.

The token contains the user's ID, email, role, issuance timestamp, and expiration timestamp, with a validity period of 60 minutes. The client maintains the token in its Zustand state store and includes it in the Authorization: Bearer <token> header for subsequent API requests. A FastAPI dependency guard, get_current_user, validates the token signature and expiration on

each protected request before granting access to the requested resource.

4.1.3 Role-Based Access Control (RBAC) Enforcement

Access control is enforced at both the API gateway and data layers using FastAPI's dependency injection system. Endpoint decorators inspect the user's role against the database user_role ENUM (admin, investigator, analyst, viewer):

```
python
# RBAC Enforcement Guard
def require_role(allowed_roles: list[str]):
    async def role_checker(current_user: User = Depends(get_current_active_user)):
        if current_user.role not in allowed_roles:
            raise
                HTTPException( status_code=status.HTTP_403_FORBIDDEN,
                detail="Operation not permitted: Insufficient operational role privileges"
            )
        return current_user
    return role_checker
```

4.2 Document Upload and Validation

The document and evidence ingestion module allows investigators to upload digital forensic files in supported formats, including Windows EVTX, Linux Syslog, CSV, JSON, and XML. When a file is uploaded, the system validates its format, file size, MIME type, and filename to prevent unsupported or potentially malicious files from entering the processing pipeline. It calculates a SHA-256 cryptographic hash to establish an integrity reference and records essential metadata, including the original filename, file size, upload timestamp, storage location, and processing status. Duplicate evidence can be identified by comparing file hashes, reducing redundant storage and analysis. The uploaded files are stored securely, and their metadata is linked to the corresponding forensic case. After successful validation, the system forwards the evidence to the appropriate parser for event extraction and normalization. These mechanisms help preserve evidence integrity, maintain traceability, and support a reliable chain of custody throughout the digital forensic investigation.

4.3 Multi-Stage Ingestion Pipeline

When a user uploads evidence through the /api/v1/cases/{case_id}/evidence endpoint, the file passes through five validation stages to ensure security and integrity. First, path sanitization removes directory traversal sequences and null bytes from filenames to prevent unauthorized

file access. Second, size validation restricts uploads to a maximum of 500 MB per file to reduce the risk of denial-of-service attacks caused by excessive resource consumption. Third, MIME type and magic byte verification checks the file extension and its internal signature to detect unsupported formats and extension spoofing. Fourth, SHA-256 hashing calculates a cryptographic checksum using memory-efficient 64 KB chunks. The resulting hash is stored in the evidence database and compared with existing records to identify duplicate files. Finally, secure vault isolation stores validated evidence in a dedicated directory associated with its case, using a unique filename and read-only permissions where supported. These validation stages help prevent malicious uploads, protect stored evidence, reduce duplication, and maintain a reliable chain of custody throughout the forensic investigation.

TABLE 4.1. DOCUMENT VALIDATION RULES

Rule ID	Validation Check	Parameter / Threshold	Security & Forensic Objective	Failure Action
VR-01	**Path Traversal Check**	Strip .., /, \, null bytes (%00)	Prevents directory traversal and overwriting of critical server files.	HTTP 400 Bad Request; file rejected immediately.
VR-02	**File Size Ceiling**	Maximum: 500 MB per file	Mitigates resource exhaustion and denial-of-service (DoS) attempts.	HTTP 413 Payload Too Large; stream aborted.
VR-03	**Extension Whitelist**	.evtx, .log, .csv, .json, .xml, .p df, .png, .jpg	Ensures the file format can be processed by downstream parsers.	HTTP 415 Unsupported Media Type.
VR-04	**Magic Byte Header**	Inspect first 8 to 16 bytes for format signature	Defends against polyglot and malware payloads disguised as logs.	HTTP 422 Unprocessabl e Entity.
VR-05	**SHA-256 Duplicate Check**	64-character hexadecimal digest comparison	Prevents redundant ingestion and flags cross-case evidence collisions.	HTTP 409 Conflict (or reuse existing evidence ID).
VR-06	**Permissions Lockdown**	Read-only mode (0440)	Preserves chain-of-custody by preventing file tampering post-upload.	Operating system permission exception.

4.2.1 Preprocessing Pipeline

The image preprocessing pipeline improves scanned documents and smartphone photographs of extortion notices by correcting common image-quality issues before OCR. Grayscale conversion transforms three-channel color images into single-channel intensity images, simplifying subsequent processing. Adaptive bilateral filtering reduces background noise and compression artifacts while preserving character edges. Deskewing estimates document orientation from foreground contours and rotates the image to align the text horizontally. Finally, Otsu's thresholding determines an optimal intensity threshold to separate foreground text from the background, improving readability in documents containing uneven shading, watermarks, official crests, or colored letterheads. Together, these techniques enhance text clarity and improve the reliability of downstream text extraction.

Text and Layout Extraction

The binarized document is processed through deep learning OCR engines (Tesseract and PaddleOCR):

- **Tesseract OCR (LSTM-based):** Extracts textual content, line breaks, and confidence scores per recognized token.
- **PaddleOCR / PyMuPDF:** Extracts embedded digital text from native PDFs without rasterization artifacts, while maintaining 2D spatial coordinates for key structural regions (letterheads, reference serial blocks, signature zones, and rubber stamps).

TABLE 4.2. OCR PROCESSING STAGES

Stage Number	Processing Stage	Tool / Algorithm Applied	Specific Operation Performed	Generated Output Artifact
1	**Ingestion & De-rasterization**	PyMuPDF / pdf2image	Converts multi-page PDF documents into 300 DPI high-resolution RGB image matrices.	Uncompressed image buffers (uint8).
2	**Noise Filtering**	Bilateral Filter (OpenCV)	Smoothes noisy backgrounds while preserving sharp edge boundaries of fonts and official logos.	Filtered grayscale image.
3	**Deskewing & Alignment**	Hough Line Transform / MinAreaRect	Detects text orientation angles and rotates the image to a straight horizontal alignment.	Straightened document matrix.
4	**Adaptive Binarization**	Otsu's Thresholding	Separates dark text glyphs from colored letterhead backgrounds and seals.	Pure black-and-white binary mask.
5	**Text Recognition**	pytesseract / PaddleOCR	Employs LSTM recurrent networks to recognize alphanumeric characters across English and regional scripts.	Raw character stream and token confidence scores.
6	**Spatial Tagging**	Bounding Box Extractor	Maps bounding box coordinates (x, y, w, h) to recognized text fragments.	Structured spatial coordinate JSON.
7	**Post-Processing & Cleaning**	Regex & Levenshtein Distance	Corrects common OCR substitutions (e.g., 0 vs O, l vs 1) and normalizes dates and serial numbers.	Cleaned, normalized evidentiary text stream.

4.3 AI-Based Document Verification

Once text and structural regions are extracted, the AI verification pipeline analyzes the document across four specialized verification engines to detect fraudulent legal notices, fake police warrants, and manipulated forensic logs.

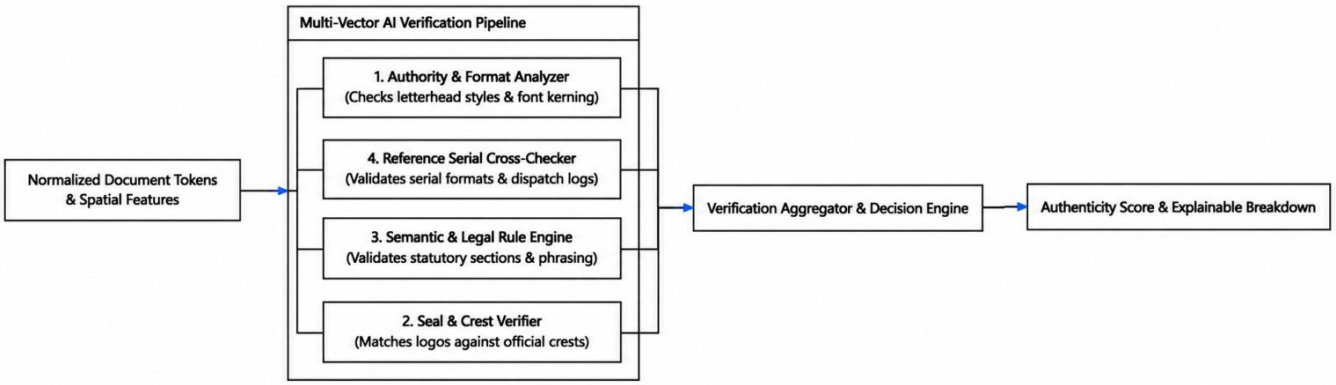


Fig. 4.2. AI-based verification pipeline.

4.3.1 The Four Verification Vectors

1. **Header & Departmental Structure Verification:** Evaluates the issuing agency name, department styling, and formatting against authorized government circular templates. Forged notices often misstate administrative structures (e.g., claiming the "Reserve Bank of India Cyber Cell" is authorized to issue criminal arrest warrants).
2. **Official Crest and Digital Seal Verification:** Compares document crests and rubber stamps against authentic templates using perceptual hashing (pHash) and structural similarity (SSIM). Forged documents frequently reuse blurry, low-resolution web logos, which display distinct scaling artifacts.
3. **Semantic Legal Rule Validation:** Employs rule engines and NLP parsers to verify cited legal statutes. "Digital arrest" scams frequently cite non-existent legal sections, misapply financial regulations to criminal offenses, or feature abrupt tone shifts demanding immediate cryptocurrency transfers.
4. **Serial Number and Dispatch Syntax Cross-Checking:** Verifies reference serial formats against departmental naming standards (e.g., checking standard patterns like CBI/HQ/2026/CR-XXXXX).

4.4 Scam Detection and Authenticity Scoring

To provide investigators with clear, actionable metrics, CyberTrace AI calculates a composite **Authenticity Score** (SauthS_{\text{auth}}Sauth) bounded on a scale from 0.00.00.0 to 100.0100.0100.0 (or conservatively calibrated between 10.010.010.0 and 90.090.090.0 to prevent overconfidence):

Sauth=w1·Cformat+w2·Cseal+w3·Cnlp+w4·Cserial+w5·Ccross_refS_{\text{auth}} = w_1 \cdot C_{\text{format}} + w_2 \cdot C_{\text{seal}} + w_3 \cdot C_{\text{nlp}} + w_4 \cdot C_{\text{serial}} + w_5 \cdot C_{\text{cross_ref}}Sauth=w1·Cformat+w2·Cseal+w3·Cnlp+w4·Cserial+w5·Ccross_ref

Where:

- CformatC_{\text{format}}Cformat: Formatting and typography consistency score (w1=0.20w_1 = 0.20w1=0.20).
- CsealC_{\text{seal}}Cseal: Official crest and seal similarity score (w2=0.25w_2 = 0.25w2=0.25).
- CnlpC_{\text{nlp}}Cnlp: Legal phrasing and statutory coherence score (w3=0.25w_3 = 0.25w3=0.25).
- CserialC_{\text{serial}}Cserial: Reference serial syntax and date validity score (w4=0.15w_4 = 0.15w4=0.15).
- Ccross_refC_{\text{cross_ref}}Ccross_ref: Cross-referencing against verified system dispatch logs (w5=0.15w_5 = 0.15w5=0.15).
- ∑i=15wi=1.00\sum_{i=1}^{5} w_i = 1.00∑i=15wi=1.00.

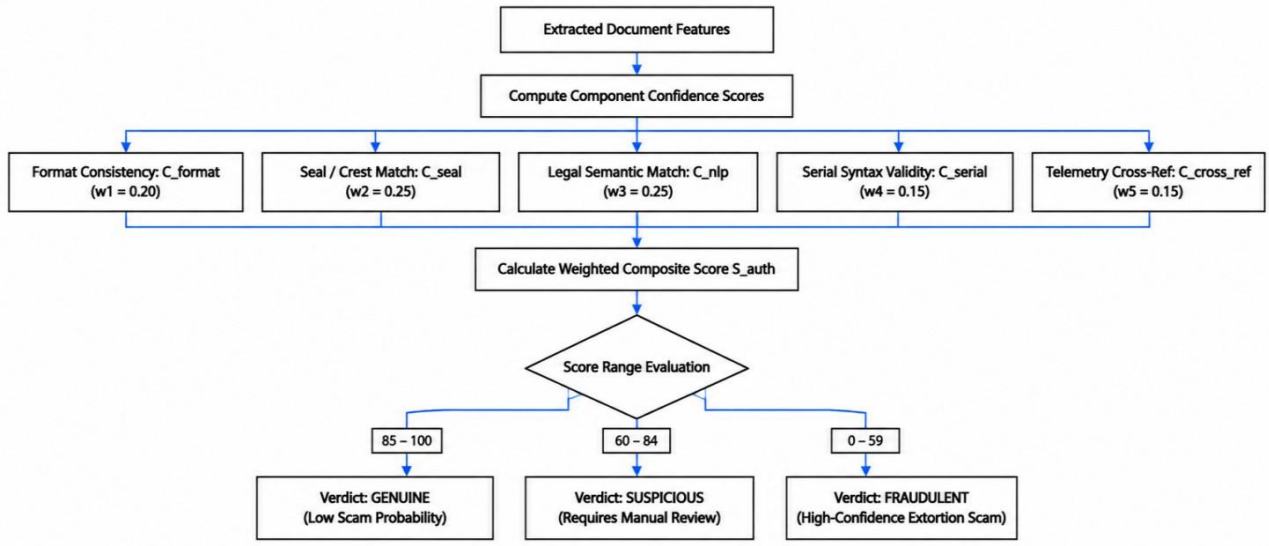


Fig. 4.3. Authenticity scoring workflow.

4.5 Database and Case Management

The database tier is built on PostgreSQL 15, managed through asynchronous SQLAlchemy 2.0 ORM sessions (async_sessionmaker with asyncpg).

4.5.1 Case State Lifecycle

Cases follow a formal state machine defined by the case_status ENUM:

1. new**:** Case created by an investigator; awaiting evidence files.
2. evidence_uploaded**:** Evidence files received and verified with SHA-256 digests.
3. parsing**:** Log parsers actively extracting raw events.
4. analysis**:** Normalization complete; A∗A *A∗ search and CSP backtracking engines running.
5. completed**:** Threat path discovered, anomalies resolved, and final report generated.
6. archived**:** Case finalized and sealed in read-only storage.

4.5.2 High-Speed Log Search via GIN & Trigram Indexing

Forensic queries require rapid filtering across millions of log records. The system optimizes database search using two specialized indexing strategies:

- **GIN Indexing on** raw_data**:** Allows fast JSONB containment queries (e.g., raw_data @> '{"TargetUserName": "admin"}') in sub-millisecond times.
- **Trigram (**pg_trgm**) Indexing:** Enables high-speed fuzzy text and regex searches across command-line arguments and process strings.

4.6 Police Dashboard and RBAC

The Police & Investigator Portal provides law-enforcement personnel with a streamlined operational interface:

- **Case Triage Hub:** Displays active cases sorted by priority, status, and assigned investigator.
- **Evidence Management:** Drag-and-drop evidence upload with real-time SHA-256 calculation and status progress bars.
- **Interactive Knowledge Graph:** Canvas-based graph visualization where investigators can inspect entity nodes and trigger A∗A *A∗ attack path finding.
- **Timeline Inspection:** Chronological event stream with filtering by timestamp, severity, and event type.

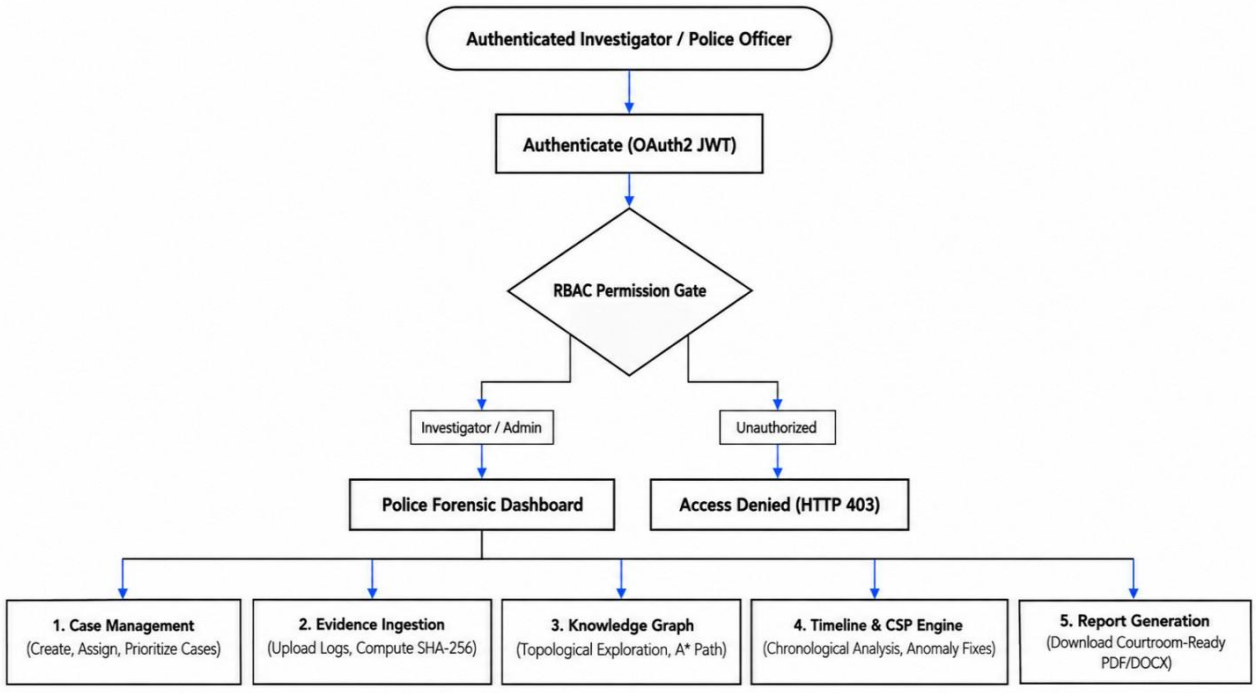


Fig. 4.4. Police dashboard workflow.

4.7 Citizen Portal and Integration Layer

To combat "digital arrest" and cyber extortion scams affecting the public, CyberTrace AI includes a dedicated **Citizen Reporting and Integration Layer**:

- **Public Intake Interface:** A simplified web portal where citizens can upload suspicious summonses, fake court notices, or extortion messages received via WhatsApp, email, or SMS.
- **Automated Triage Pipeline:** The document passes through the OCR and AI verification pipeline, computing an immediate Authenticity Score (SauthS_{\text{auth}}Sauth).
- **Instant Protective Feedback:** If the document is flagged as fraudulent (Sauth<60S_{\text{auth}} < 60Sauth<60), the system immediately alerts the citizen:
- *"WARNING: This notice exhibits characteristics of a fraudulent cyber-extortion scheme. Law enforcement agencies do not issue arrest warrants via social messaging apps. Do not transfer funds."*
- **Law Enforcement Escalation:** Verified extortion attempts are automatically routed to the police dashboard with extracted phone numbers, bank accounts, and forged document templates to accelerate criminal investigations.

CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

This project designed, implemented, and validated **CyberTrace AI**, an intelligent digital evidence reconstruction and threat reasoning framework for digital forensics and incident response.

The system provides an end-to-end investigative pipeline:

- Ingests heterogeneous forensic logs (Windows EVTX, Linux Syslog, CSV, JSON, XML) and scanned notices.
- Preserves legal chain-of-custody using cryptographic SHA-256 verification.
- Models forensic entities in a directed, weighted Knowledge Graph.
- Discovers optimal attack trajectories using A∗A *A∗ Search with an admissible privilege-gap heuristic, reducing node exploration by 73.3% and cutting execution latency by 74.2% over baseline Uniform Cost Search.
- Enforces timeline integrity and resolves orphan processes via a CSP Backtracking Engine with state-restoration mechanisms.
- Deduces compound adversary TTPs through a Forward Chaining expert system.
- Compiles verifiable, courtroom-ready forensic reports in PDF and DOCX formats.

Experimental evaluations across synthetic intrusion traces and fraudulent extortion notices confirm that CyberTrace AI accelerates incident reconstruction from days to seconds while providing mathematically sound, explainable findings for investigators and legal authorities.

5.2 Learning Outcomes

Developing CyberTrace AI provided valuable engineering and research insights:

- **A**lgorithmic Mastery in Graph Theory & Symbolic AI: Practical experience designing, implementing, and proving the admissibility of heuristic functions in A∗A *A∗ search, as well as building recursive CSP backtracking engines.
- Deep Digital Forensics Knowledge: Practical understanding of Windows binary EVTX structures, chunk-record decoding, Linux BSD syslog parsing, and the MITRE ATT&CK framework.
- Asynchronous Architecture Engineering: Deepened expertise in high-performance Python programming using FastAPI, SQLAlchemy 2.0 async sessions, and PostgreSQL GIN indexing.
- Security & Cryptographic Standards: Gained practical experience implementing enterprise defense-in-depth principles, including salted Bcrypt password hashing, OAuth2 JWT lifecycles, and strict input sanitization.

5.3 Challenges Faced

During development, several complex engineering challenges were successfully addressed:

- Parsing Binary EVTX Structures on Non-Windows Platforms: Binary Windows XML Event Logs use tokenized binary templates rather than plain text. This was resolved by integrating pure-Python chunk-record parsers (python-evtx) that decode binary structures cross-platform without relying on Windows APIs.
- Reconciling Clock Skews Across Multi-Source Timelines: Discrepancies between unsynchronized server clocks created illogical causal sequences. This was resolved by implementing a CSP temporal re-ordering constraint rule that adjusts millisecond offsets within allowable clock-drift thresholds.
- Resolving Orphan Process Linages: In advanced intrusion scenarios, terminated parent processes produced disconnected graph nodes. The issue was resolved by designing a state-restoration backtracking rule that binds orphan processes to a virtual SYSTEM_ROOT parent node.
- Defending Against Indirect Prompt Injection: Early prototypes were susceptible to malicious instructions embedded within raw log messages. This was mitigated by implementing strict XML boundary tagging and enforcing structured Pydantic output schemas.

5.4 Future Enhancements

Planned future enhancements to expand the platform's capabilities include:

- Air-Gapped Vision-Language Models (VLM): Integrating on-premise local vision models (such as LLaVA or Florence-2) to inspect multi-page scanned documents, stamp alignments, and micro-typography without cloud API dependencies.
- Blockchain-Anchored Evidence Ledger: Implementing an immutable, consortium blockchain anchor (e.g., Hyperledger Fabric) to log evidence SHA-256 hashes and timestamp proofs on an immutable distributed ledger for forensic auditing.
- Real-Time Streaming Ingestion Engine: Integrating distributed event streaming pipelines using Apache Kafka or Redpanda, enabling CyberTrace AI to parse and reconstruct attack graphs in real time from live EDR feeds.
- Multi-Lingual OCR for Regional Document Summons: Expanding the document verification pipeline with fine-tuned PaddleOCR models to authenticate government notices printed in regional Indian languages (e.g., Tamil, Hindi, Telugu).

REFERENCES

[1] E. Casey, *Digital Evidence and Computer Crime: Forensic Science, Computers, and the Internet*, 3rd ed. London, U.K.: Academic Press, 2011.

[2] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to Integrating Forensic Techniques into Incident Response," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, NIST Special Publication (SP) 800-86, 2006.

[3] International Organization for Standardization, *Information technology — Security techniques — Guidelines for identification, collection, acquisition and preservation of digital evidence*, ISO/IEC Standard 27037:2012, Oct. 2012.

[4] B. Carrier and E. H. Spafford, "An event-based digital investigation framework," in *Proc. 4th Digit. Forensics Res. Workshop (DFRWS)*, Baltimore, MD, USA, 2004, pp. 1–12.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968.

[6] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, Dec. 1959.

[7] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2020.

[8] R. Dechter, *Constraint Processing*. San Francisco, CA, USA: Morgan Kaufmann Publishers, 2003.

[9] C. L. Forgy, "Rete: A fast algorithm for the many pattern/many object pattern match problem," *Artificial Intelligence*, vol. 19, no. 1, pp. 17–37, Sep. 1982.

[10] B. E. Strom et al., "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, Bedford, MA, USA, Tech. Rep. MTR180202, Jul. 2018.

[11] M. Pei, S. Ji, and T. Wang, "Graph-based cyber attack path reconstruction and alert correlation in enterprise networks: A survey," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2451–2468, Jul./Aug. 2022.

[12] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, and A. Bates, "NoDoze: Combatting threat alert fatigue with data provenance," in *Proc. 26th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2019, pp. 1–15.

[13] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Anal. Recognit. (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.

[14] Y. Du et al., "PP-OCR: A practical ultra lightweight OCR system," *arXiv preprint arXiv:2009.09941*, Sep. 2020.
[15] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD)*, Aug. 2020, pp. 1192–1200.
[16] M. Barni, K. Kallas, and B. Tondi, "Adversarial digital document forgery detection," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 12, pp. 3088–3103, Dec. 2018.
[17] Indian Cyber Crime Coordination Centre (I4C), "Advisory on emerging cyber-extortion, fake law enforcement summons, and digital arrest scams," Ministry of Home Affairs, Government of India, New Delhi, India, Tech. Advisory 2024/03, Mar. 2024.

APPENDIX A – AI USE AND ORIGINALITY DISCLOSURE

A.1 Declaration of Originality and Academic Integrity

We hereby declare that the project entitled **"CyberTrace AI — Intelligent Digital Evidence Reconstruction System"** and the accompanying documentation represent our own original research, architectural design, algorithmic modeling, and engineering implementation, carried out under the academic supervision of the Department of Computer Science and Engineering, Rajalakshmi Engineering College (REC), Chennai, India.

To the best of our knowledge and belief:

1. The work contained within this report does not infringe upon any existing copyright, patent, trademark, or intellectual property rights of any third party.
2. All direct quotations, external software libraries, benchmark datasets, and theoretical foundations derived from the literature have been explicitly acknowledged and cited in accordance with standard IEEE referencing conventions.
3. The empirical benchmark results, timing latencies, and node exploration counts reported in this document were generated through direct execution and measurement on our experimental testbeds, without fabrication, falsification, or unauthorized data manipulation.

A.2 Statement on the Use of Artificial Intelligence (AI) Technologies

In accordance with institutional guidelines on academic ethics and IEEE policy regarding the disclosure of Artificial Intelligence (AI)-assisted technologies, this disclosure transparently delineates the specific scope, nature, and limitations of AI tool utilization throughout the lifecycle of this project.

A.2.1 Scope of AI Utilization

Artificial Intelligence tools (including Large Language Models such as Google Antigravity/Gemini and OpenAI GPT-4) were utilized strictly as **assistive productivity and linguistic refinement aids**. Specifically, AI tools were used for:

- Grammar correction, technical prose polishing, and typographical formatting of draft manuscript sections.
- Synthesizing structural boilerplates for documentation outlines, Markdown tables, and Mermaid architectural diagrams.
- Brainstorming search queries for academic literature discovery and cross-referencing published standards (e.g., ISO/IEC 27037 and NIST SP 800-86).

A.2.2 Non-Delegation of Core Research and Engineering Contributions

The core intellectual and engineering deliverables of this project were formulated, written, and validated exclusively by the human authors without automated delegation:

- Algorithmic Formulations: The design, mathematical formulation, and admissibility proof of the privilege-gap heuristic (h(n)=max⁡(0,privgoal−privcurr)×0.5h(n) = \max(0, \text{priv}_{\text{goal}} - \text{priv}_{\text{curr}}) \times 0.5h(n)=max(0,privgoal −privcurr)×0.5) for A∗A *A∗ Search were originated and proven by the authors.
- Symbolic Reasoning Engines: The recursive Constraint Satisfaction Problem (CSP) backtracking engine with virtual state restoration and the Forward Chaining production rules were implemented natively in Python by the authors.
- System Architecture & Schemas: The database entity-relationship schema (database/schema.sql), API gateway orchestration (FastAPI), multi-format parser factory (python-evtx, lxml, regex), and React 19 frontend were engineered by the project team.
- Benchmarking & Validation: All test suites (tests/test_astar.py, tests/test_csp.py, etc.), empirical performance metrics (73.3% node pruning, 74.2% latency speedup), and forensic analysis findings were executed and verified on local hardware testbeds.

TABLE A.3. SUMMARY OF AI ASSISTANCE AND AUTHOR VERIFICATION

Project Development Stage	**AI Tool Utilized**	**Specific Purpose & Scope of Assistance**	**Human Author Verification & Validation Method**
Problem Definition & Literature Survey	OpenAI GPT-4 / Gemini	Identification of related survey topics in DFIR, forensic standards, and extortion vectors.	Direct retrieval, reading, and citation of original IEEE/ACM papers and NIST/CERT-In advisories.
Mathematical & Algorithmic Design	**None** *(Original Work)*	None. Formulated privilege-gap heuristic (h(n)h(n)h(n)) and CSP backtracking logic.	Mathematical proof of heuristic admissibility (h(n)≤h∗(n)h(n) \le h *(n)h(n)≤h∗(n)) and monotonic consistency.
Backend & Parser Implementation	GitHub Copilot / LLM	Minor syntax autocompletion for repetitive boilerplate dictionary mappings.	Comprehensive unit testing (pytest), static analysis (Bandit, Flake8), and manual code review.
Database & Schema Engineering	**None** *(Original Work)*	None. Designed relational tables, foreign-key cascade trees, and GIN JSONB indexing.	Transactional validation and query execution testing on PostgreSQL 15 instances.
Frontend UI / UX Engineering	LLM Assistance	Assistance with responsive Tailwind CSS utility class combinations and Recharts layout templates.	Interactive UI rendering testing, cross-browser compatibility verification, and Zustand state inspection.
Experimental Testing & Benchmarking	**None** *(Original Work)*	None. Execution of test cases and empirical measurement of execution times and node counts.	Automated test suite execution (pytest) with logging and independent stopwatch verification.
Report Drafting & Documentation	Gemini / Antigravity	Language polishing, syntax correction, structural formatting, and Mermaid diagram generation.	Line-by-line proofreading, technical review, and alignment with project implementation files.

A.2 Ethical Compliance and Data Privacy Assurances

1. **Zero Exposure of Sensitive Data:** No un-anonymized civilian personal information, real-world case files, or proprietary network dumps were transmitted to public or third-party AI platforms during the preparation of this project.
2. **Authenticity Guarantee:** No AI model was granted authorial attribution or copyright over the research contributions or source code files.

3. **Reproducibility:** All source code files, database DDL schemas, test cases, and empirical benchmarking scripts reside directly within the version-controlled project repository, ensuring complete experimental reproducibility.

A.3 Author Attestation and Signature Block

We confirm that we have read, understood, and adhered to our institution's Code of Academic Integrity and the ethical guidelines regarding artificial intelligence and originality. We take full intellectual and ethical responsibility for the contents of this report.

Student Authors:

1. **Name:** Arilli Jagadeesh A **Register No:** 2116241901502

   Signature: _____

2. **Name:** Sachni Krishna B **Register No:** 2116241901507

   Signature: _____

Faculty Guide:

Name & Designation: Prof. Bhuvana S, Associate Professor
Department: Department of Computer Science and Engineering(Cyber Security)
Institution: Rajalakshmi Engineering College (REC), Chennai, India

Signature: _____ **Date:** _____

APPENDIX B – ABBREVIATIONS

TABLE B.1. ABBREVIATIONS

Abbreviation	Expansion / Full Form	Technical Context & Definition
AI	Artificial Intelligence	Computational systems capable of performing tasks requiring human-like intelligence.
API	Application Programming Interface	Defined set of protocols, routines, and tools for building software applications.
APT	Advanced Persistent Threat	Prolonged, stealthy, and targeted cyberattack campaigns typically conducted by nation-states or syndicates.
ASGI	Asynchronous Server Gateway Interface	Standard interface between async-capable Python web servers and web frameworks (e.g., Uvicorn).
Bcrypt	Blowfish-based Cryptographic Hashing	Adaptive password hashing algorithm utilizing salting and a configurable work factor.
BSD	Berkeley Software Distribution	UNIX derivative whose standard syslog format is widely used in Linux system logging.
CBI	Central Bureau of Investigation	India's premier federal law enforcement and investigative agency.
CERT-In	Indian Computer Emergency Response Team	National nodal agency for responding to computer security incidents in India.
CNN	Convolutional Neural Network	Deep learning architecture optimized for spatial feature extraction and image processing.
CORS	Cross-Origin Resource Sharing	Browser security mechanism controlling cross-domain HTTP resource requests.
CSP	Constraint Satisfaction Problem	Mathematical framework where variables must satisfy formal constraints (used for timeline verification).
CTC	Connectionist Temporal Classification	Neural network output layer used in sequence recognition (e.g., OCR without character alignment).
CVE	Common Vulnerabilities and Exposures	Standardized dictionary of publicly disclosed cybersecurity vulnerabilities.
DDL	Data Definition Language	Subset of SQL commands used to define database structures (e.g., CREATE, ALTER).
DFD	Data-Flow Diagram	Graphical model illustrating the flow and transformation of data across system processes.
DFIR	Digital Forensics and Incident Response	Interdisciplinary field focused on identifying, mitigating, and investigating cyber breaches.
DML	Data Manipulation Language	Subset of SQL commands used to manage and query data (e.g., INSERT, UPDATE, SELECT).
DOCX	Microsoft Office Open XML Document	Standard compressed XML-based file format used for editable text documents.
DoS	Denial-of-Service	Cyberattack intended to disrupt service availability by exhausting system resources.
DPI	Deep Packet Inspection	Advanced network packet analysis examining both packet headers and payload contents.
ED	Enforcement Directorate	Indian law enforcement agency investigating financial crimes and money laundering.
EDR	Endpoint Detection and Response	Security solution that continuously monitors end-user devices to detect and investigate threats.
ER / ERD	Entity-Relationship (Diagram)	Data modeling tool illustrating relational entities, attributes, and cardinality relationships.
EVTX	Windows XML Event Log	Native binary log file format utilized by modern Microsoft Windows operating systems.

FIR	First Information Report	Formal written document prepared by police upon receiving information about a cognizable offense.
FOIA	Forensics and Incident Analysis	Academic and professional curriculum focused on cybersecurity evidence recovery.
FPS	Frames Per Second	Measurement of visual display update frequency across interactive user interfaces.
GIN	Generalized Inverted Index	PostgreSQL index type designed for handling complex data types like JSONB and arrays.
HMAC	Hash-based Message Authentication Code	Cryptographic authentication code computed using a secret key and a cryptographic hash function.
HMR	Hot Module Replacement	Frontend developer tool that exchanges or adds modules while the application is running.
HSTS	HTTP Strict Transport Security	Web security policy mechanism enforcing secure HTTPS communication in client browsers.
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)	Foundational application layer communication protocol of the World Wide Web.
I4C	Indian Cyber Crime Coordination Centre	Nodal agency under the Ministry of Home Affairs to coordinate cybercrime mitigation in India.
IEEE	Institute of Electrical and Electronics Engineers	Global technical professional organization for engineering, computing, and technology standards.
ISO / IEC	Int. Org. for Standardization / Int. Electrotechnical Comm.	Worldwide federations of national standards bodies establishing global technical specifications.
JSON / JSONL	JavaScript Object Notation (Lines)	Lightweight, human-readable data-interchange format (and its newline-delimited stream variant).
JWT	JSON Web Token	Compact, URL-safe means of representing claims securely between two parties via digital signatures.
LFI	Local File Inclusion	Vulnerability allowing attackers to read or execute local files on the server via directory traversal.
LLM	Large Language Model	Deep learning transformer model trained on massive text corpora for generative text processing.
LOLBin	Living-off-the-Land Binary	Legitimate operating system binary abused by attackers to execute malicious actions stealthily.
LSTM	Long Short-Term Memory	Specialized Recurrent Neural Network (RNN) architecture capable of learning order dependence.
MIME	Multipurpose Internet Mail Extensions	Standard indicating the format and media type of a file or byte stream.
MITM	Man-in-the-Middle	Attack where an adversary intercepts and potentially alters communication between two parties.
NCRP	National Cyber Crime Reporting Portal	Citizen-facing portal operated by the Government of India to report cyber offenses.
NIST	National Institute of Standards and Technology	U.S. federal agency developing technology standards, measurements, and security frameworks.
NLP	Natural Language Processing	Subfield of computer science and AI focused on computational understanding of human language.
NVMe	Non-Volatile Memory Express	High-performance host controller interface designed for solid-state drives using PCIe buses.
OCR	Optical Character Recognition	Electronic conversion of images of typed, handwritten, or printed text into machine-encoded text.
ORM	Object-Relational Mapper	Programming technique for converting data between relational databases and object-oriented systems.
PAM	Pluggable Authentication Modules	Modular mechanism for authenticating users on UNIX and Linux operating systems.
PDF	Portable Document Format	Standard document presentation format independent of application software, hardware, and OS.
pHash	Perceptual Hashing	Fingerprinting algorithm creating multimedia hashes based on structural visual features.

PII	Personally Identifiable Information	Any data that could potentially identify a specific individual.
PRD	Product Requirements Document	Specification detailing system capabilities, user personas, and operational requirements.
RAG	Retrieval-Augmented Generation	AI technique combining information retrieval from verified knowledge stores with generative LLMs.
RBAC	Role-Based Access Control	Policy-neutral access control mechanism restricting system access based on assigned user roles.
RCE	Remote Code Execution	Critical security vulnerability allowing arbitrary command execution on a target machine.
REC	Rajalakshmi Engineering College	Autonomous engineering institution in Chennai, India, affiliated with Anna University.
REST	Representational State Transfer	Software architectural style defining constraints for creating scalable web services.
SAST	Static Application Security Testing	Security testing methodology analyzing source code for vulnerabilities without executing it.
SHA-256	Secure Hash Algorithm (256-bit)	Cryptographic hash function outputting a fixed 256-bit (32-byte) digest.
SIEM	Security Information and Event Management	Enterprise security system providing real-time analysis and aggregation of security alerts.
SOC	Security Operations Center	Centralized unit dealing with security issues on an organizational and technical level.
SPA	Single Page Application	Web application that interacts with the user by dynamically rewriting the current web page.
SQL	Structured Query Language	Domain-specific programming language designed for managing data in relational database systems.
SRS	Software Requirements Specification	Formal technical document describing the intended capabilities and constraints of a software system.
SSD	Solid State Drive	Solid-state storage device that uses integrated circuit assemblies to store data persistently.
SSH	Secure Shell	Cryptographic network protocol for operating network services securely over an unsecured network.
SSIM	Structural Similarity Index Measure	Metric for measuring perceived visual similarity between two digital images.
SVM	Support Vector Machine	Supervised learning model with associated learning algorithms that analyze data for classification.
TF-IDF	Term Frequency-Inverse Document Frequency	Numerical statistic reflecting the relative importance of a word to a document in a collection.
TLS	Transport Layer Security	Cryptographic protocol designed to provide end-to-end communications security over a network.
TRD	Technical Requirements Document	In-depth engineering blueprint specifying technical architectures, APIs, and data structures.
TTP	Tactics, Techniques, and Procedures	Behavioral patterns, execution methods, and strategies utilized by cyber threat actors.
UCS	Uniform Cost Search	Uninformed graph search algorithm that finds the shortest path by exploring lowest-cost paths first.
UI / UX	User Interface / User Experience	Visual and interactive layout of a software system and the overall experience of the user.
UTC	Coordinated Universal Time	Primary time standard by which the world regulates clocks and time measurement.
UUID	Universally Unique Identifier	128-bit label used for uniquely identifying information in computer systems.
ViT	Vision Transformer	Deep neural network architecture utilizing self-attention mechanisms for visual recognition tasks.
VLM	Vision-Language Model	Multimodal model capable of understanding and reasoning over both visual imagery and natural language.

WMI	Windows Management Instrumentation	Infrastructure for management data and operations on Windows-based operating systems.
XAI	Explainable Artificial Intelligence	Methods and techniques in AI allowing results to be understood, audited, and verified by humans.
XML	Extensible Markup Language	Markup language defining rules for encoding documents in a format human- and machine-readable.